

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине «Алгоритмы и структуры данных»
Тема: «Создание кольцевого буфера»

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Иванов Д.В.

Санкт-Петербург

2025

СОДЕРЖАНИЕ

1.	Цель работы	3
2.	Задание	4
3.	Теоретический минимум	6
4.	Выполнение работы	7
4.1.	Описание структуры кода	7
4.2.	Описание пайплайна	10
5.	Исследование	11
6.	Вывод	15
	Приложение А. Код программы	16
	Приложение Б. Тестирование	19

1. ЦЕЛЬ РАБОТЫ

Создать кольцевой буфер для целых чисел на основе статического массива на языке Python, который поддерживает методы вставки в начало и конец, удаления из начала и конца, проверки на пустоту и заполненность, а также подсчета текущего размера. Исследовать скорость работы каждого метода, построить графики зависимости времени выполнения операций от емкости дека.

2. ЗАДАНИЕ

Требуется реализовать кольцевой буфер (а именно кольцевую деку) для целых чисел на основе статического массива длины N . Реализация должна представлять собой класс *CircularDeque*.

В конструкторе класс должен принимать количество элементов N в массиве, а также значение *bool push_force*, отражающее, будут ли при переполнении буфера элементы перезаписывать старые.

Класс должен поддерживать следующие методы (каждый за $O(1)$ в худшем случае или амортизированно):

- *void push_front(x)* – добавить элемент x в начало деки;
- *void push_back(x)* – добавить элемент x в конец деки;

Если дека заполнена, а *push_force* указан как *false*, при попытке добавить элементы в деку должно выбрасываться исключение; если же *push_force* указан как *true*, то при добавлении элемента в конец деки он должен записаться поверх элемента с начала деки, а при добавлении элемента в начало – поверх элемента с конца. При этом начало/конец деки должно сместиться на последующий/предыдущий элемент.

Также необходимо реализовать следующие методы:

- *int pop_front()* – удалить и вернуть элемент из начала деки;
- *int pop_back()* – удалить и вернуть элемент из конца деки;
- *int front()* – вернуть (не удаляя сам элемент) значение первого элемента в деке;
- *int back()* – вернуть значение последнего элемента;
- *int size()* – вернуть текущее число элементов в деке;
- *bool empty()* – проверить деку на пустоту;
- *bool full()* – проверить деку на заполненность (если массив полон).

- `void resize(new_capacity)` – изменить размер статического массива и перезаписать элементы в массив нового размера. При перезаписи необходимо перемещать элементы в правильном порядке - от начала деки к её концу. Если не все элементы старого массива помещаются в новый, необходимо оставить лишь *new_capacity* первых (с начала деки) элементов массива.

Если выполнить какую-либо операцию невозможно, необходимо выбросить исключение.

3. ТЕОРЕТИЧЕСКИЙ МИНИМУМ

Дек — абстрактный тип данных, основанный на двусторонней очереди, который позволяет добавлять и удалять элементы как с начала, так и с конца. Рассматриваемый дек образует кольцо: после последнего элемента идет первый. При записи в полный дек новые элементы заменяют старые, что делает возможным избежание потери пространства и помогает экономить ресурсы памяти за счет статического размера.

Кольцевой дек поддерживает операции вставки элементов в начало и конец. Их асимптотическая сложность равна $O(1)$, так как изначально известны голова и хвост дека, а также его размер остается неизменным. Операции удаления элементов из начала и конца также имеют константную асимптотическую сложность по тем же причинам. Сложность операции расширения емкости кольцевого дека составляет $O(n)$. Это обусловлено тем, что в заново выделенную область памяти требуется перезаписать n элементов.

4. ВЫПОЛНЕНИЕ РАБОТЫ

4.1. Описание структуры кода

Реализован класс *CircularDeque*, включающий следующие поля:

- *list self.deque* — статический массив, хранящий элементы кольцевого дека;
- *bool self.push_force* — поле, отражающее возможность записи новых элементов в полный дек;
- *int self.n* — заданная емкость дека;
- *int self.cur_size* — текущий размер дека;
- *int self.head* — индекс головы дека;
- *int self.tail* — индекс хвоста дека;

Конструктор принимает на вход емкость дека и возможность перезаписи старых элементов на новые при переполнении. Если заданная емкость отрицательна, вызывается исключение *ValueError*.

Класс содержит следующие методы:

- *def push_front(self, x: int)* — метод вставки в начало дека;

Принимаемые аргументы:

x: int — вставляемый элемент.

- *def push_back(self, x: int)* — метод вставки в конец дека;

Принимаемые аргументы:

x: int — вставляемый элемент.

- *def pop_front(self) -> int* — метод удаления элемента из начала дека;

Возвращаемое значение:

Возвращает значение удаленного элемента.

- *def pop_back(self) -> int* — метод удаления элемента из конца дека;

Возвращаемое значение:

Возвращает значение удаленного элемента.

- *def front(self) -> int* — метод, возвращающий значение первого элемента дека;

- *def back(self) -> int* — метод, возвращающий значение последнего элемента дека;
- *def size(self) -> int* — метод, возвращающий текущий размер дека;
- *def empty(self) -> bool* — метод, проверяющий дек на пустоту;

Возвращаемое значение:

True, если дек пустой, иначе *False*.

- *def full(self) -> bool* — метод, проверяющий дек на заполненность;

Возвращаемое значение:

True, если дек заполнен, иначе *False*.

- *def resize(self, new_cap: int)* – метод, изменяющий емкость дека.

Принимаемые аргументы:

new_cap: int – новая емкость дека.

Описание принципов работы основных методов:

1. Метод вставки в начало: первоначально дек проверяется на заполненность. Если есть возможность записи в заполненный дек и заданная емкость дека не равна нулю, происходит сдвиг индекса хвоста на один влево, иначе вызывается исключение *OverflowError*. После проверки индекс головы сдвигается на один влево, на его место в дек записывается вставляемый элемент. В конце текущий размер дека увеличивается на единицу, в случае переполнения становится равным установленной ранее емкости.
2. Метод вставки в конец: логика аналогична методу вставки в начало, за исключением того, что при заполненности дека индекс головы сдвигается на один вправо, после проверки индекс хвоста сдвигается на один вправо и на его место вставляется новый элемент.
3. Метод удаления из начала: проверяется текущий размер дека. Если он не равен нулю, то в отдельной переменной запоминается удаляемый элемент, после этого его значение в деке заменяется на 0, текущий размер уменьшается на единицу. Если после уменьшения размер ненулевой, голова сдвигается на один вправо, если же дек оказался

пустым, то индексы головы и хвоста становятся равными нулю. В итоге метод возвращает удаленное значение. Если дек изначально был пуст, вызывается исключение *IndexError*.

4. Метод удаления из конца: логика аналогична методу удаления из начала, за исключением того, что вместо головы в отдельной переменной запоминается хвост, и затем его индекс сдвигается на один влево в случае, если дек не пуст.
5. Метод изменения емкости: при новой емкости большей нуля создается статический массив данной емкости. Далее происходит сравнение новой емкости и текущего размера дека. Если первая меньше, то индекс хвоста становится равен новой емкости, уменьшенной на единицу, текущий размер дека приравнивается к значению новой емкости. Иначе индекс хвоста равняется текущему размеру, уменьшенному на единицу. Далее проверяется случай расширения для пустого дека: индексы головы и хвоста устанавливаются в 0, старое значение емкости меняется на новое, создается статический массив из нулей, и метод завершает работу. В случае, если дек не был пустым, происходит вставка элементов из старого дека в новый, соблюдая последовательность от головы к хвосту. Индекс головы становится равным 0, индекс хвоста равняется посчитанному индексу в первом вложенном блоке ветвления, емкость изменяется на новую, в поле массива записывается новый дек. При новой емкости равной нулю все поля устанавливаются в 0, на место дека помещается пустой массив. В противном случае вызывается исключение *ValueError*.

4.2. Описание пайплайна

Данные, а именно емкость дека и возможность перезаписи старых элементов, поступают в экземпляр класса при его создании. Далее, используя полученные значения, в классе создается дек, который изменяется в зависимости от выбранного метода (в него вставляются новые элементы, или

удаляются старые, или дек изменяет свою емкость). Другие данные, такие как индекс головы/хвоста и текущий размер, хранятся в полях класса, описанных в разделе 4.1. При обращении к конкретному полю экземпляра возвращаются запрашиваемые значения, которые впоследствии можно вывести с помощью функции `print`.

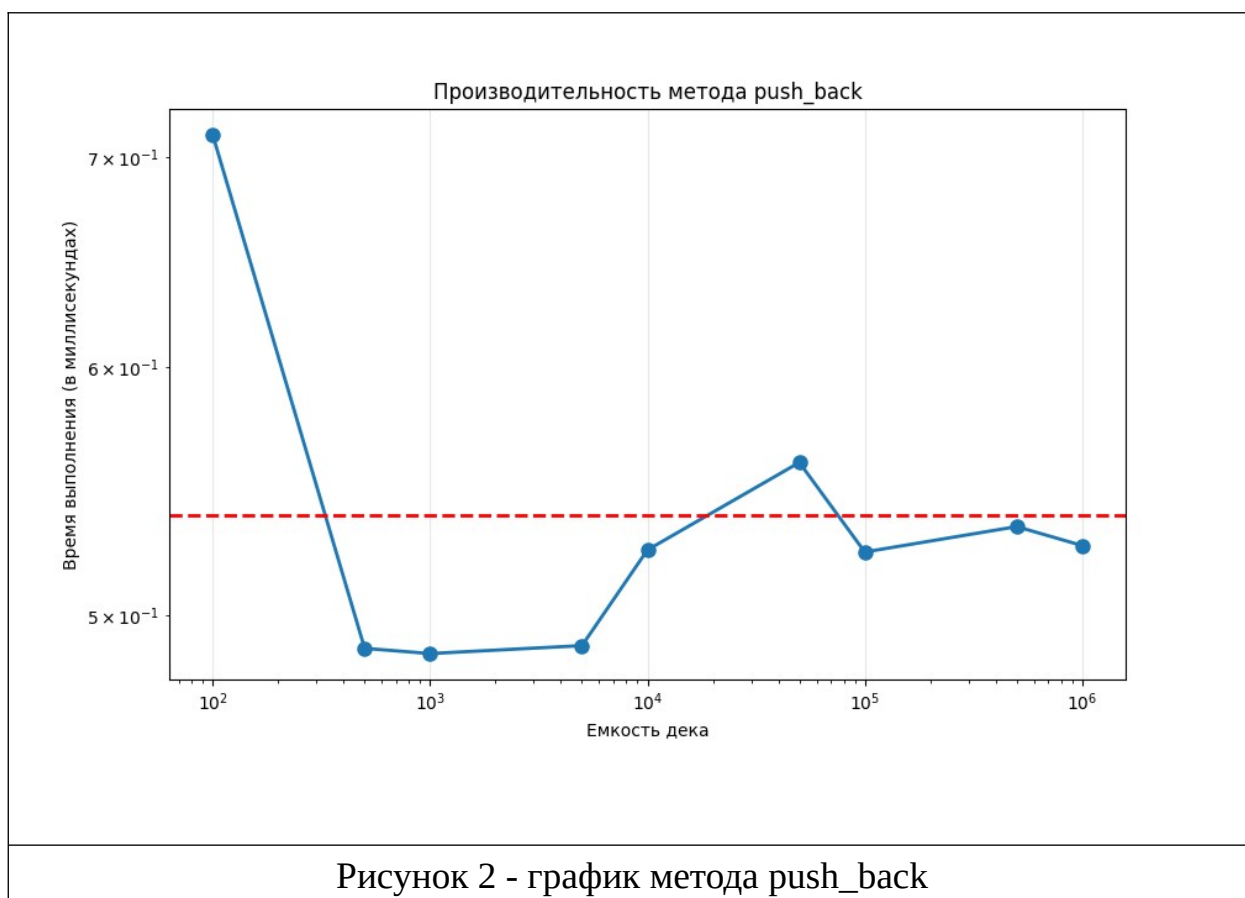
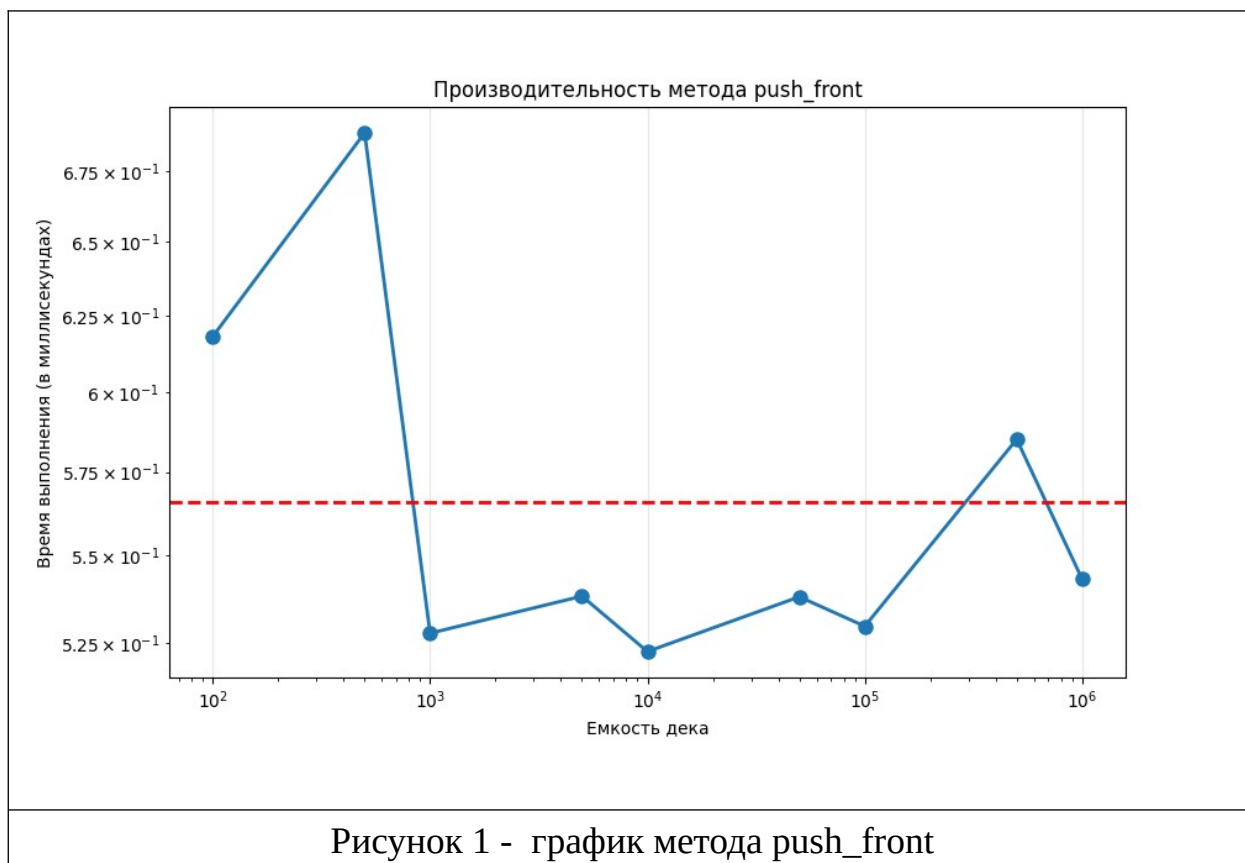
ИССЛЕДОВАНИЕ

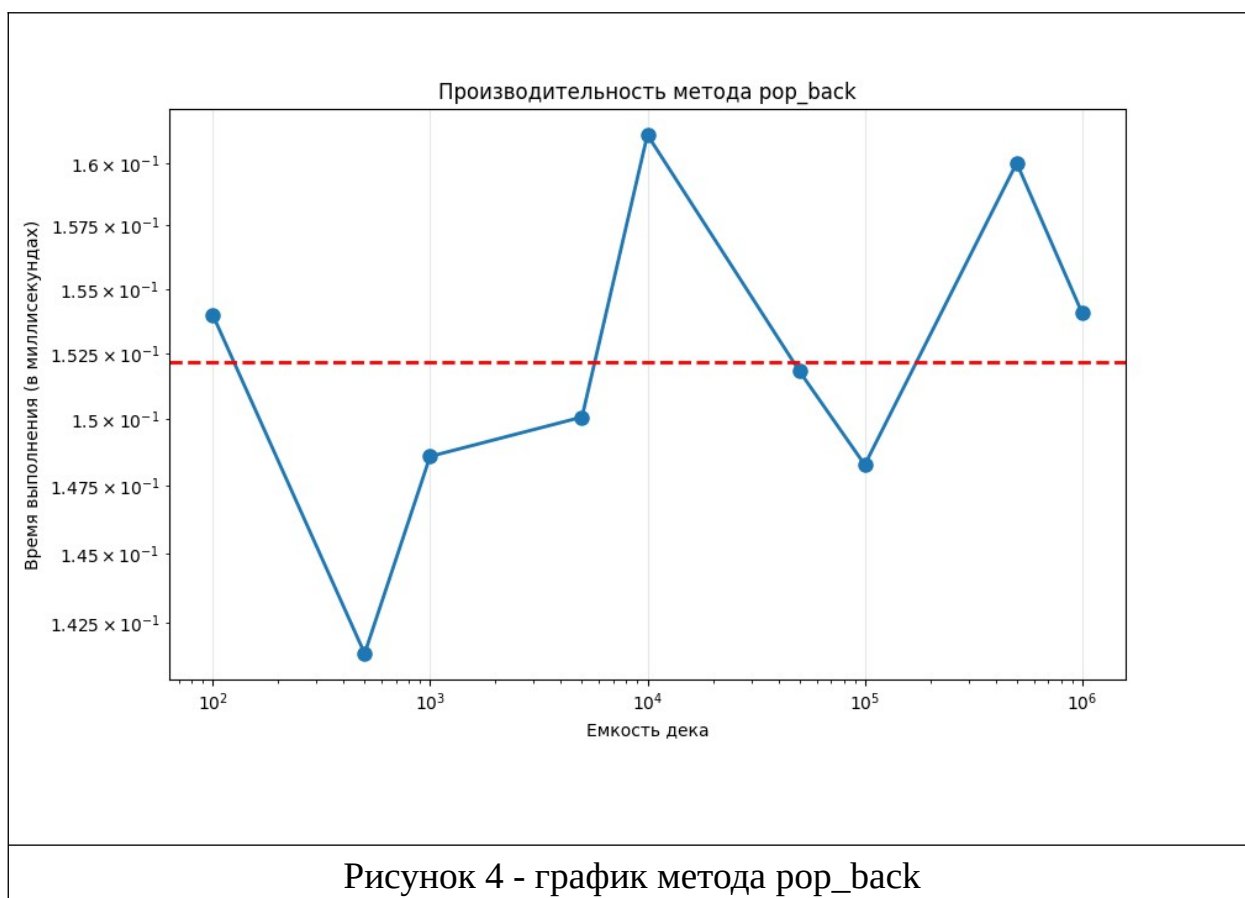
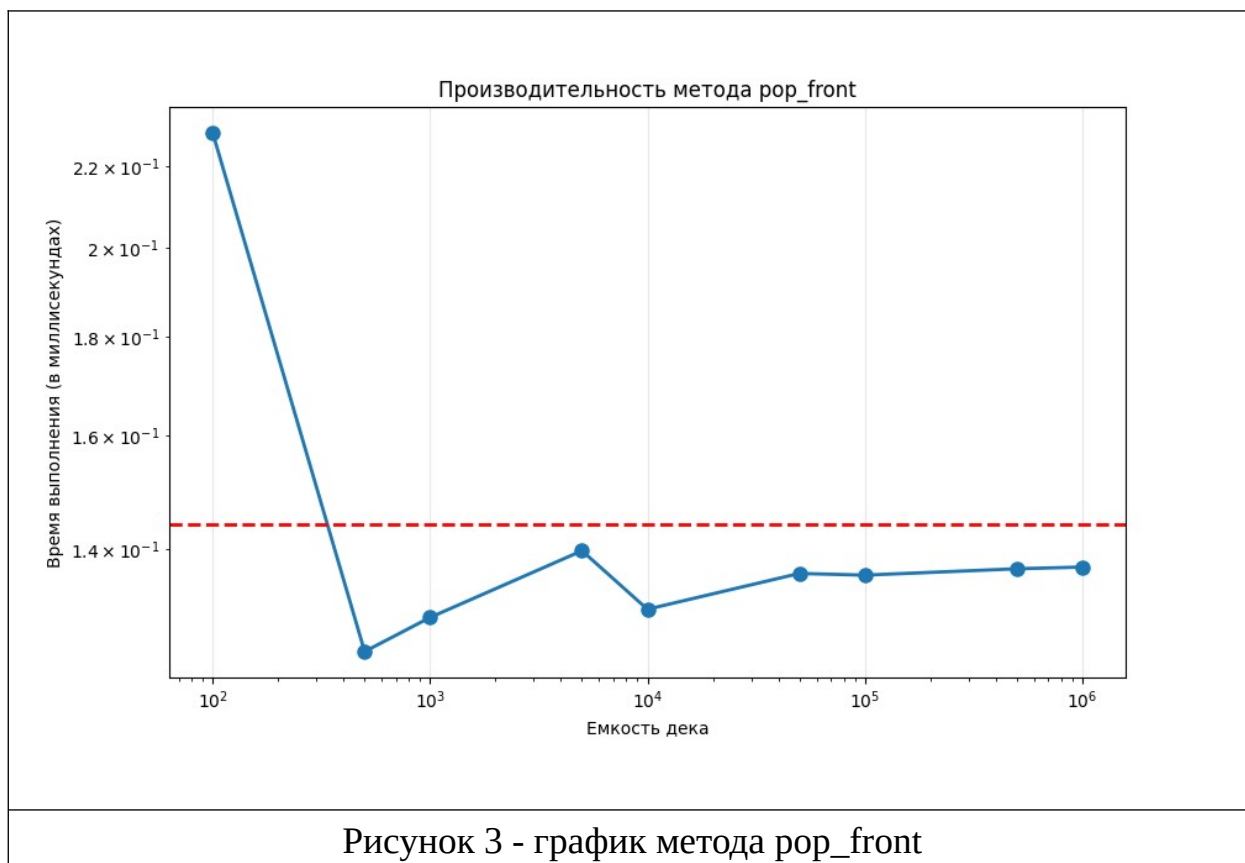
Для оценки асимптотической сложности каждого метода были проведены исследования их работы на различных объемах данных (от 100 до 1000000 случайных элементов в диапазоне от -1000 до 1000).

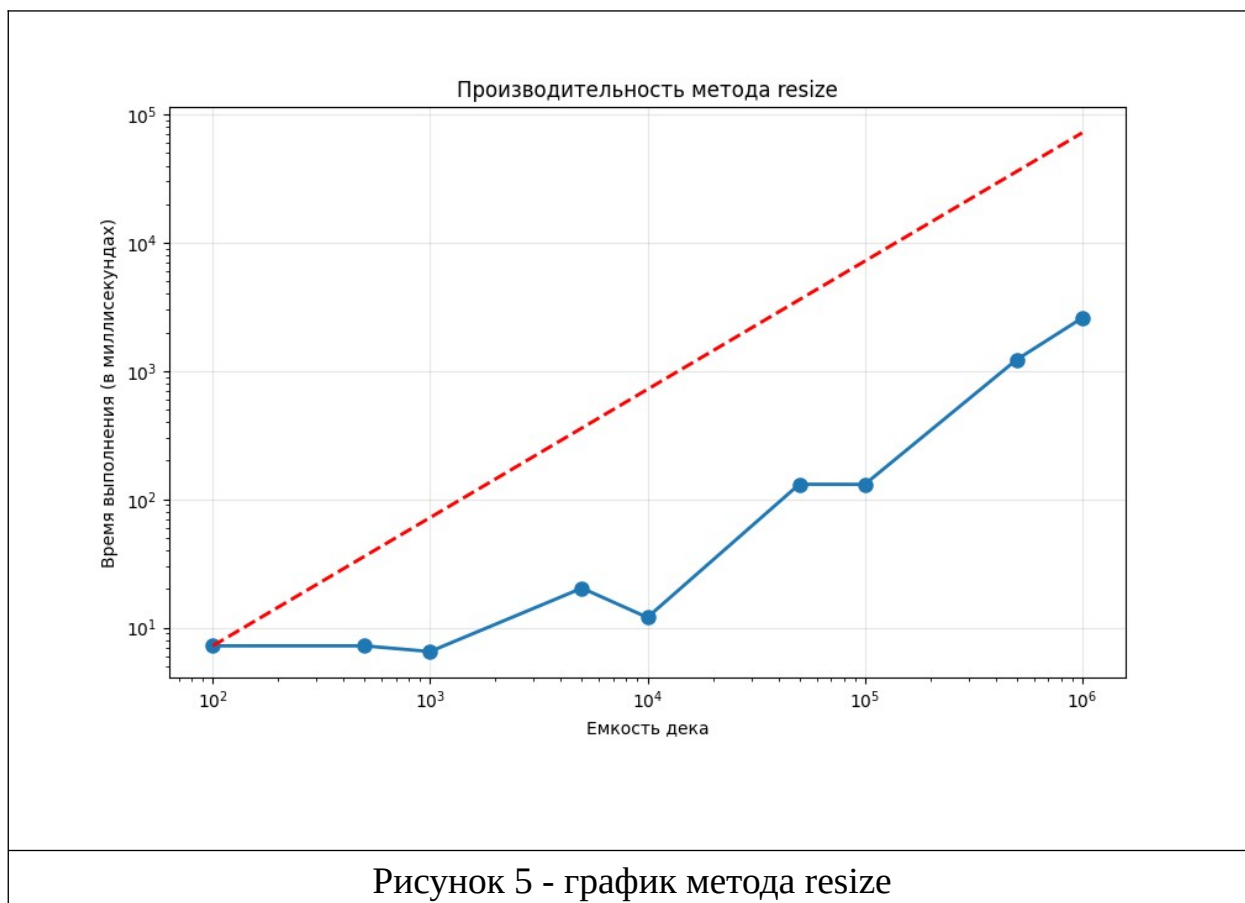
Для методов *push_front* и *push_back* измеряется время вставки одного элемента при разных емкостях дека. При построении графиков (см. рис. 1, рис. 2) полученные значения колеблются в пределах $O(1)$, поэтому сложность методов оценивается как константная. Также такая сложность объясняется тем, что перед вставкой заранее известны голова и хвост, поэтому нет необходимости проходиться по всем элементам.

Перед замером времени для методов *pop_front* и *pop_back* создаются заполненные дека, после этого происходит удаление элементов и отслеживание скорости этих операций. Так как заранее известны индексы головы и хвоста, то сложность операций составляет $O(1)$. Теоретическую сложность подтверждают полученные графики (см. рис. 3, рис. 4): точки находятся в окрестности константной прямой.

До замера времени метода *resize* создается заполненный дек некоторой длины, далее измеряется время работы одного вызова *resize* для различных емкостей. Полученные значения времени увеличиваются с ростом размера дека. Так происходит, потому что выделяется память под новую емкость и перезаписываются старые элементы. Сложность $O(n)$ подтверждается на графике (см. рис. 5), время растет линейно.







6. ВЫВОД

В результате выполнения лабораторной работы реализован класс, отражающий принципы работы кольцевого дека. За основу взят статический массив заданной емкости, который заполнялся целыми числами. С учетом возможности замены старых элементов новыми созданы методы вставки и удаления из начала и конца дека. Реализован метод изменения емкости как на меньшую, так и на большую относительно начальной, сохраняющий последовательность элементов от головы к хвосту. Прделана работа с исключениями: в случае, если какую-либо операцию выполнить невозможно, применяется инструкция *raise* и программа завершает работу. Также исследована производительность каждого метода, построены графики оценки сложности операций. Теоретически рассчитанная временная сложность совпадает со значениями на графиках, что подтверждает правильность вычислений.

ПРИЛОЖЕНИЕ А

КОД ПРОГРАММЫ

```
class CircularDeque:
    def __init__(self, n: int, push_force=False):
        if n < 0:
            raise ValueError("Wrong deque capacity")
        self.deque = [0] * n
        self.push_force = push_force
        self.n = n
        self.cur_size = 0
        self.head = 0
        self.tail = 0

    def push_front(self, x: int):
        if self.full():
            if self.push_force and self.n != 0:
                self.tail = (self.n + (self.tail - 1)) % self.n
            else:
                raise OverflowError("Deque is overflowed")

            self.head = (self.n + (self.head - 1)) % self.n if
self.cur_size else self.head
            self.deque[self.head] = x

            self.cur_size = min(self.n, self.cur_size+1)

    def push_back(self, x: int):
        if self.full():
            if self.push_force and self.n != 0:
                self.head = (self.head + 1) % self.n
            else:
                raise OverflowError("Deque is overflowed")

            self.tail = (self.tail + 1) % self.n if self.cur_size else
self.tail
            self.deque[self.tail] = x

            self.cur_size = min(self.n, self.cur_size + 1)

    def pop_front(self) -> int:
        if self.cur_size != 0:
            popped_head = self.deque[self.head]
            self.deque[self.head] = 0
            self.cur_size -= 1
            if self.cur_size != 0:
                self.head = (self.head + 1) % self.n
            else:
                self.tail = 0
                self.head = 0
            return popped_head
        else:
            raise IndexError("Deque is empty")

    def pop_back(self) -> int:
```



```

    if self.cur_size != 0:
        popped_tail = self.deque[self.tail]
        self.deque[self.tail] = 0
        self.cur_size -= 1
        if self.cur_size != 0:
            self.tail = (self.n + (self.tail - 1)) % self.n
        else:
            self.tail = 0
            self.head = 0
        return popped_tail
    else:
        raise IndexError("Deque is empty")

def front(self) -> int:
    if self.empty():
        raise IndexError("Deque is empty")
    return self.deque[self.head]

def back(self) -> int:
    if self.empty():
        raise IndexError("Deque is empty")
    return self.deque[self.tail]

def size(self) -> int:
    return self.cur_size

def empty(self) -> bool:
    return True if self.cur_size == 0 else False

def full(self):
    return True if self.cur_size == self.n else False

def resize(self, new_cap: int):
    if new_cap > 0:
        new_deque = [0] * new_cap
        head = self.head
        tail = self.tail

        if new_cap < self.cur_size:
            tail = new_cap - 1
            self.cur_size = new_cap
        elif new_cap >= self.cur_size:
            tail = self.cur_size - 1

        if self.cur_size == 0:
            self.head = 0
            self.tail = 0
            self.n = new_cap
            self.deque = [0] * new_cap
            return

        for i in range(tail):
            new_deque[i] = self.deque[head]
            head = (head + 1) % self.n

        new_deque[tail] = self.deque[head]

        self.head = 0

```

```
        self.tail = tail
        self.n = new_cap
        self.deque = new_deque

    elif new_cap == 0:
        self.head = 0
        self.tail = 0
        self.cur_size = 0
        self.n = 0
        self.deque = []

    else:
        raise ValueError("Wrong new capacity")
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

```
import pytest
from lab_1 import CircularDeque

def test_init():
    with pytest.raises(ValueError):
        CircularDeque(-1, False)

def test_push_front_in_empty_deque():
    deq = CircularDeque(0, True)
    with pytest.raises(OverflowError):
        deq.push_front(1)

def test_push_front_with_push_force_false():
    deq = CircularDeque(1, False)
    deq.push_front(1)
    with pytest.raises(OverflowError):
        deq.push_front(2)

def test_push_front_basic():
    deq = CircularDeque(2, True)
    deq.push_front(1)
    deq.push_front(2)
    deq.push_front(3)
    assert deq.cur_size == deq.n

def test_push_back_in_empty_deque():
    deq = CircularDeque(0, True)
    with pytest.raises(OverflowError):
        deq.push_back(1)

def test_push_back__with_push_force_false():
    deq = CircularDeque(1, False)
    deq.push_back(1)
    with pytest.raises(OverflowError):
        deq.push_back(2)

def test_push_back_basic():
    deq = CircularDeque(1, True)
    deq.push_back(1)
    deq.push_back(2)
    assert deq.cur_size == deq.n

def test_pop_front():
    deq = CircularDeque(1, False)
```

```

    with pytest.raises(IndexError):
        deq.pop_front()

    deq.push_back(1)
    deq.pop_front()
    assert deq.cur_size == 0

def test_pop_back():
    deq = CircularDeque(1, False)
    with pytest.raises(IndexError):
        deq.pop_back()

    deq.push_back(1)
    deq.pop_front()
    assert deq.cur_size == 0

def test_front_and_back():
    deq = CircularDeque(2, True)
    deq.push_back(1)
    assert deq.front() == 1
    assert deq.back() == 1
    deq.push_back(2)
    assert deq.front() == 1
    assert deq.back() == 2

def test_front_and_back_in_empty_deque():
    deq = CircularDeque(1, True)
    with pytest.raises(IndexError):
        deq.front()
        deq.back()

def test_size():
    deq = CircularDeque(1, True)
    assert deq.size() == 0
    deq.push_front(1)
    assert deq.size() == 1
    deq.pop_front()
    assert deq.size() == 0

def test_empty():
    deq = CircularDeque(1, True)
    assert deq.empty()
    deq.push_back(1)
    assert not deq.empty()

def test_full():
    deq = CircularDeque(1, True)
    assert not deq.full()
    deq.push_back(1)
    assert deq.full()

```

```

def test_resize_from_null():
    deq = CircularDeque(0, True)
    deq.resize(3)
    assert deq.n == 3

def test_resize_more():
    deq = CircularDeque(2, True)
    deq.push_front(1)
    deq.push_front(2)
    head_before = deq.front()
    tail_before = deq.back()
    deq.resize(4)
    assert deq.front() == head_before and deq.back() == tail_before
    assert deq.deque[deq.tail + 1] == 0

def test_resize_less():
    deq = CircularDeque(2, True)
    deq.push_front(1)
    deq.push_front(2)
    head_before = deq.front()
    tail_before = deq.tail
    size_before = deq.size()
    deq.resize(1)
    assert deq.front() == head_before and deq.back() ==
deq.deque[tail_before-1]
    assert deq.size() == size_before - 1

def test_resize_to_null():
    deq = CircularDeque(2, True)
    deq.push_front(1)
    deq.push_front(2)
    deq.resize(0)
    assert deq.size() == 0 and deq.n == 0

def test_push_pop_push():
    deq = CircularDeque(3, True)
    for i in range(3):
        deq.push_back(i)
    deq_before = deq.deque
    for i in range(3):
        deq.pop_back()
    for i in range(3):
        deq.push_back(i)
    assert deq.deque == deq_before

def test_push_force_false():
    deq = CircularDeque(2, False)
    deq.push_front(1)
    deq.push_front(2)
    with pytest.raises(OverflowError):
        deq.push_back(3)

```

```
def test_push_force_true():
    deq = CircularDeque(2, True)
    deq.push_front(1)
    deq.push_front(2)
    deq.push_back(3)
    assert deq.size() == 2 and deq.deque == [1, 3]
    assert deq.front() == 1
```